Project 2: EC2_CONTAINERIZATION_AND_EBS_DEPLOYMENT

Wednesday, December 17, 2025 12:35 AM

INFRASTRUCTURE

- **VPC (**project-EBS-vpc**)**

  Used to setup the private network Architecture. It has the following features:

1. IPV4 CIDR block **--> 10.0.0.0/16**
2. IPV6 CIDR block **--> None (** *We don't need one in this Project* **)**
3. Tenancy **--> Default (** *we don't need a dedicated Hardware for the to be created EC2 Instance* **)**
4. AZ **--> 1**
5. Public Subnet **--> 1**
6. Private Subnet **--> 1**
7. Public Subnet CIDR block **--> 10.0.0.0/20**
8. Private Subnet CIDR block **--> 10.0.128.0/20**
9. NAT Gateways --> None for Now (*it will be created later*)
10. VPC Endpoints **--> None (** *they will be created later* **)**
11. DNS Options **--> DNS Hostnames and Resolution enabled (** *Needed for the creation and assessment of DNS Hostnames in the public subnet* **) .**

- **Public-Subnet (**project-EBS-subnet-public1-us-east-1a**)**

  Used for the creation of the **EC2 Instance** that will be created **during the deployment of the App in Elastic Beanstalk**. The Nat Gateway will also be created in it to enable the **Instance in the**

private subnet to reach the Internet. It has the following features:

1. Network ACL **--> All traffic is allowed at the inbound and outbound rules** (*to avoid access problem in and out the subnet* **).**
2. Route table **-->** It has a route **(***destination***: VPC CIDR) to guide traffic flow to the VPC Network as a whole** using the **local** *target***,** and another route **to the Internet** using the **Internet Gateway** *target.*

- **Private-Subnet (**project-EBS-subnet-private1-us-east-1a**)**

Used for the creation of the **EC2 Instance** that will be used to build the **web App** and test it with **Docker**. The Nat Gateway will also be created in it to enable the **Instance in the private subnet to reach the Internet** and **download NGINX latest version**. It has the following features:

1. Network ACL **--> All traffic is allowed at the inbound and outbound rules** (*to avoid access problem in and out the subnet* **).**
2. Route table **-->** It has a route **(***destination***: VPC CIDR) to guide traffic flow to the VPC Network as a whole** using the **local** *target***,** and another route **to anywhere in the VPC** using the **NAT Gateway** *target (* in our scenario *it will guide the traffic from the private subnet (EC2 Instance) to the Internet Gateway in the Public subnet for it to reach the Internet) .* It also has a 3rd route that will **guide traffic to the S3 prefix ID** using the **S3 Endpoint** (Gateway) to enable traffic from the EC2 Instance to access Amazon S3.

- **EndPoints**

Used for resources accessibility across the VPC and to the Internet. It has the following Endpoints:

1. **ssm-endpoint, ssm-messages-endpoint** and **ec2-messages-endpoint** (Interfaces) **-->** The **ssm-endpoint** and **ssm-messages-endpoint** are used to activate the connection between AWS session manager and the private EC2 Instance. This is the more secured way to do that (*since it should not be accessed through the internet* **).** The **SSM-Agent** in the Instance will get in contact with the **ssm-endpoint** to communicate with **SSM service,** while the **EC2 Instance** uses **ec2-messages-endpoint** to communicate with **SSM** through **ssm-messages-endpoint.**
2. **ecr-api-endpoint** and **ecr-dkr-endpoint** (Interfaces) **-->** Both endpoint helps to access Elastic Container registry (ECR) API, and access Docker without having to pass through the Internet (Increase security). I can now access yum/dnf repository and Docker images in ECR. I then used it to build and test (run) my Web Application in the Instance.
3. **S3-Endpoint** (Gateway) **-->** Used to connect the **EC2 Instance** to **S3** and upload the built **Web Application files** after test**,** for the deployment in **Amazon Elastic Beanstalk.**

- ## NAT Gateway

  Used to enable the **private EC2 Instance** to access **the Internet** to download the **latest version of NGINX** whose template i will then use to build my **Web Application**. It will direct traffic from the **private instance** to the **internet Gateway** in the **public subnet** since it is been created there. It takes the **private IP Address** of the Instance, changes it to the **Elastic IP Address** and uses the Elastic IP to send the traffic to the Internet.

- ## Route Tables

Used to direct traffic to different resources. It gives the route to be taken and the destination.
It has the following Tables:

1. **project-EBS-rtb-public1-us-east-1a --->** The Route Table attached to the **public subnet** and has an **internet Gateway** route as well as a **local** VPC route**.**
2. **project-EBS-rtb-private1-us-east-1a --->** The Route Table attached to the **private subnet** and has a **S3 endpoint Gateway** route as well as a **local** VPC route and a **Nat Gateway** route.

- **IAM Role**

A **SSM-Role** is created with the following permissions:

1. **AmazonSSMManagedInstanceCore --->** Give the **SSM Agent** in the Instance **permissions** to access SSM through the previously created Endpoints.
2. **UploadFromEC2ToBucket --->** Inline Policy that gives the **SSM Agent** in the Instance **permissions** to access S3 through the previously created S3 Endpoint Gateway and upload files in my Bucket.
   The Role is attached to the Private EC2 Instance.

- **Private EC2 Instances (**EBS-project-Server**)**

Used to **setup the Docker environment** that will be used to build the **web App** and test it. The **App files** will after be sent to **S3** for deployment. Since the connection to the Instance will be done through the **session manager**, there is **no** need for **key Pairs or any particular Port to be opened**, hence a default security Group with **All** Port and Protocol in Inbound and Outbound rule set. It will be created in the private subnet.

- **S3 Bucket (**ebs-project-bucket-v15901**)**

Used to **host the Web App's Html and Docker files.** They will then be used by Elastic Beanstalk to deploy the Application.

- **Elastic Beanstalk (**EBSProjectApp-env**)**

Used to **deploy the Web App** using its **Html and Docker files.** They will then be used by Elastic Beanstalk to deploy the Application.